Project 2

Vincenzo Illiano
(Dated: September 23, 2026)

github: FYS3150-Project2

PROBLEM 1

Starting with

$$\gamma \frac{d^2u(x)}{dx^2} = -Fu(x). \tag{1}$$

let's define $\hat{x} = x/L$ and let $\hat{u}(\hat{x})$ be the function such that

$$\hat{u}(\hat{x}(x)) = u(x). \tag{2}$$

Because of this property, for simplicity we will write $\hat{u}(\hat{x}) = u(\hat{x})$. Because of the chain rule we have

$$\frac{d}{dx} = \frac{d\hat{x}}{dx}\frac{d}{d\hat{x}} \implies \frac{d^2}{dx^2} = \frac{d}{dx}\left(\frac{d}{dx}\right) = \frac{d\hat{x}}{dx}\frac{d}{d\hat{x}}\left(\frac{d\hat{x}}{dx}\frac{d}{d\hat{x}}\right), \tag{3}$$

thus, by using eq. 2, the left side of eq. 1 becomes:

$$\gamma \frac{d\hat{x}}{dx}\frac{d}{d\hat{x}}\left(\frac{d\hat{x}}{dx}\frac{d}{d\hat{x}}\right)u(\hat{x}) = \frac{\gamma}{L^2}\frac{d^2u(\hat{x})}{d\hat{x}^2}, \tag{4}$$

where we used $d\hat{x}/dx = 1/L$. At the same time the right hand side of eq. 1 simply becomes $-Fu(\hat{x})$. Overall we have:

$$\frac{\gamma}{L^2}\frac{d^2u(\hat{x})}{d\hat{x}^2} = -Fu(\hat{x}),$$

which by defining $\lambda = FL^2/\gamma$ simply becomes:

$$\frac{d^2u(\hat{x})}{d\hat{x}^2} = -\lambda u(\hat{x}) \tag{5}$$

PROBLEM 2

This is mostly done in `problem2.cpp`. We can see that either using the analytic formula or using the Eigen built in function `SelfAdjointEigenSolver` yields the same result (apart for a sign difference independent of the normalization of the eigenvectors).

PROBLEM 3

(a): This is done in the function `max_offdiag_symmetric` in `project2functions.cpp`. The function assumes that the matrix is symmetric, therefore it only checks elements in the upper part of the matrix. Elements in the upper off-diagonal part of the matrix are simply cycled through in order to find the largest (absolute value) one and, in such case, update the indices `k` and `l`. In order to keep track of the largest element, the variable `toret`, initialized to zero, is updated every time that an element is larger than it is found, and, eventually, returned.

(b): This is done in `problem3.cpp`.

PROBLEM 4

(a): This is done in the function `jacobi_eigensolver.cpp` in `project2functions`. I pretty much followed the code snippet suggested. The elements of the matrix A_n and R_n are updated manually instead of simply using Eigen, in order to avoid doing more calculations than necessary.

(b): This is done in `problem4.cpp`. Setting `epsilon` $= 10^{-9}$ and `maxiter` $= 10^9$ the algorithm converges in 36 iterations. The eigenvalues given by the jacobi eigensolver match the analytical ones up to the 14-th significant digit on average. On the other hand, the components of the eigenvectors given by the jacobi eigensolver match the analytical ones up to the 10-th significant digit on average.

PROBLEM 5

(a): Let's assume that `maxiter` is large enough so that the jacobi eigensolver always converges. Then we imagine that, given a $N \times N$ tridiagonal matrix A of type $(-1, 2, -1)$, the number of iterations required by the jacobi eigensolver to diagonalize A is going to be some function of N and ϵ. We can therefore write iterations $= f(N, \epsilon)$. We can graph some of the points of this function by running the jacobi eigensolver for different values of ϵ and N. This is done in `problem5.cpp` and in `plotIterations.py`. We get the result shown in fig. 1

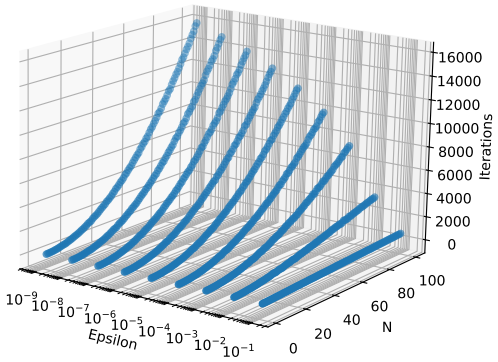


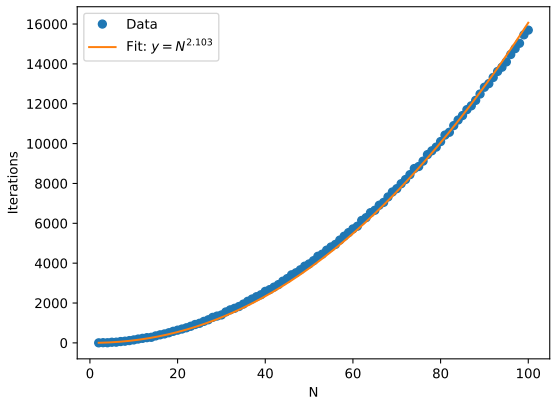
FIG. 1: Number of iterations required by the jacobi eigensolver for different values of N and ϵ when A is a tridiagonal matrix $(-1, 2, -1)$

If now we assume that ϵ is fixed at some value $\bar{\epsilon}$, we can plot the function $f(N, \bar{\epsilon})$. Roughly, it looks like $f \propto N^{\alpha}$, where α is some function of ϵ. We can therefore try to fit the function $f(N, \bar{\epsilon})$ to the function $N^{\alpha_{\bar{\epsilon}}}$, in order to find $\alpha_{\bar{\epsilon}}$. If for example we choose $\epsilon = 10^{-9}$, using `plotIterationRow.py` we get the f shown in fig. 2(a).

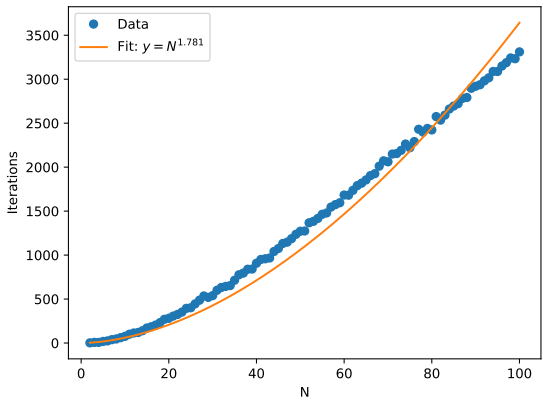
It seems that for $\epsilon = 10^{-9}$, f is proportional to $N^{2.1}$. We can do this same fit for different values of $\bar{\epsilon}$ ranging from 10^{-1} to 10^{-9}, in order to estimate how α depends on $\bar{\epsilon}$. If we do this, we get the graph shown in fig. 3.

Note however that the function x^{α} doesn't seem to be a very good fit for ϵ bigger than $\approx 10^{-2}$, as shown in fig. 2(b). However, even though this is a completely empirical observation, it seems that, for very small ϵ, $f(N)$ tends to be well approximated by x^2.

(b): We can use this same approach to determine $f(N, \epsilon)$ in the case where A is a dense matrix. Similarly to how we did in (a), we get the graph shown in fig. 4. Like we did in (a), we can choose a fixed $\bar{\epsilon}$, plot the function $f(N, \bar{\epsilon})$ and fit it to N^{α}. This is shown in fig. 5. For $\epsilon = 10^{-9}$ we get $\alpha \approx 2.131$, while in the case of tridiagonal matrix we got the slightly smaller $\alpha \approx 2.103$. Interestingly, when $\epsilon = 10^{-2}$, in the case of dense matrices (see fig. 5(b)) we seem to get a much better fit to the function N^{α} in comparison to the case of tridiagonal matrices (see fig. 2(b)). It'also interesting to notice how, when α is very small, there doesn't seem to be a huge difference in the number of iterations between the dense and the tridiagonal matrices. This is maybe due to the fact that, after every iteration, the jacobi rotation algorithm may turn 0 elements into non zero elements (while others instead become 0), making the case of a tridiagonal matrix, after a few iterations, similar to the case of dense matrices.



((a)) $f(N, \bar{\epsilon})$ for $\epsilon = 10^{-9}$



((b)) $f(N, \bar{\epsilon})$ for $\epsilon = 10^{-2}$

FIG. 2

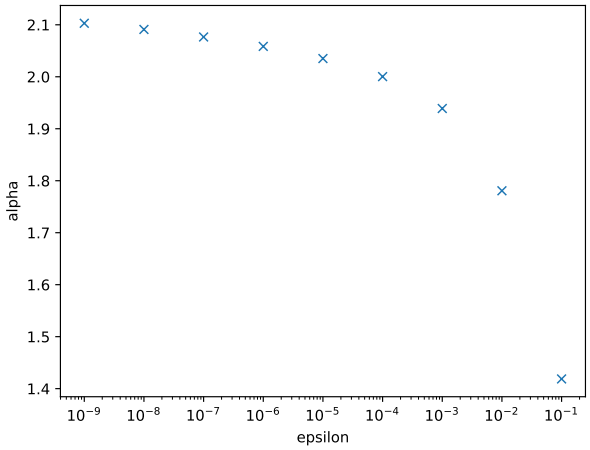


FIG. 3: Dependence of α on $\bar{\epsilon}$

PROBLEM 6

(a): The solutions (so the analytical eigenvectors and the numerical eigenvectors with Jacobi's algorithm) are calculated in `problem6.cpp` and then plotted by `plotSolution.py`. With a discretization size $n = 10$, $\epsilon = 10^{-9}$ and `maxiter` $= 10^9$ (this way the eigensolver always converges) we get the result shown in fig. 6(b).

The plot also shows the analytical eigenvectors solutions, even though at this scale these are indistinguishible from the jacobi eigensolver solutions.

(b): If we do the same with $n = 100$ we get the result shown in fig. 6(a). Once again, the analytical and the numerical solutions are not distinguishible at this scale. It's interesting that regardless of n, the analytical eigenvalues are close to the jacobi eigensolver eigenvalues up to the 10th significant digit, since $\epsilon = 10^{-9}$ (as discussed in problem 4); both the analytical and the jacobi eigensolver eigenvalues, however, change when n is changed, approaching the "correct" value as n is increased to infinity. In other words, the similarity between the analytical eigenvectors/values and the jacobi eigensolver ones only depends on ϵ, while the similarity between any of these solutions and the correct solution to the buckling beam equation also depends on n, as we would expect.

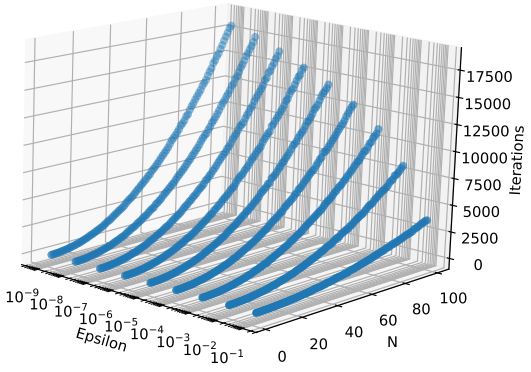
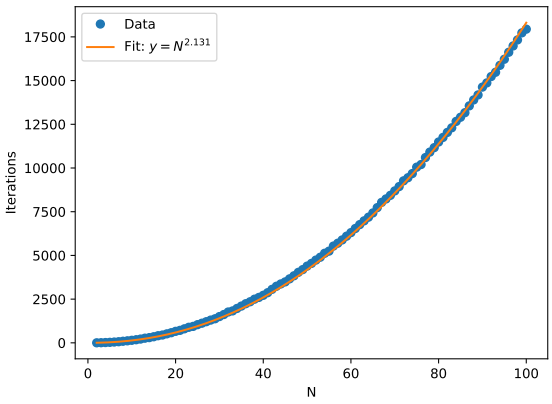
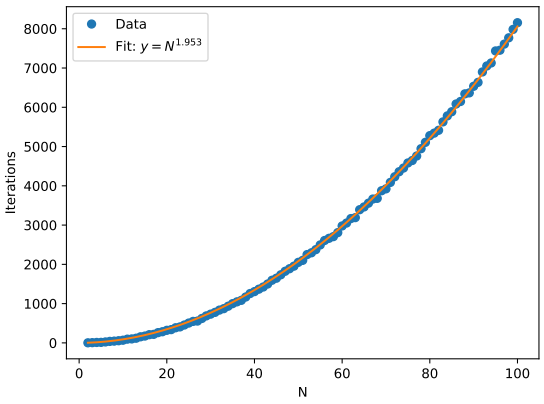


FIG. 4: Number of iterations required by the jacobi eigensolver for different values of N and ϵ when A is a random dense matrix

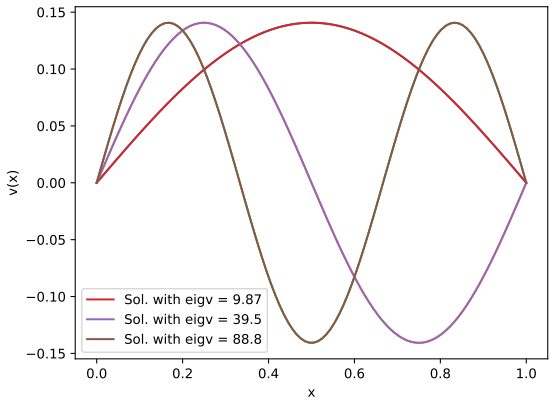


((a)) $f(N, \bar{\epsilon})$ for $\epsilon = 10^{-9}$ for dense matrices

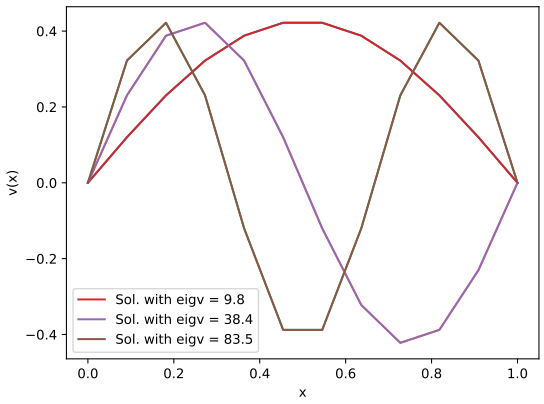


((b)) $f(N, \bar{\epsilon})$ for $\epsilon = 10^{-2}$ for dense matrices

FIG. 5



((a)) Solutions to the buckling beam equation with $n = 100$



((b)) Solutions to the buckling beam equation with $n = 10$

FIG. 6